Python 200
Week 1: Analytical Thinking

CODE THE DREAM

Welcome to Python 200

- 11 weeks. Four phases. One goal: data engineering with AI.

- Phase 1 -- Week 1: Statistical & Analytical Thinking
  - You are here ↓

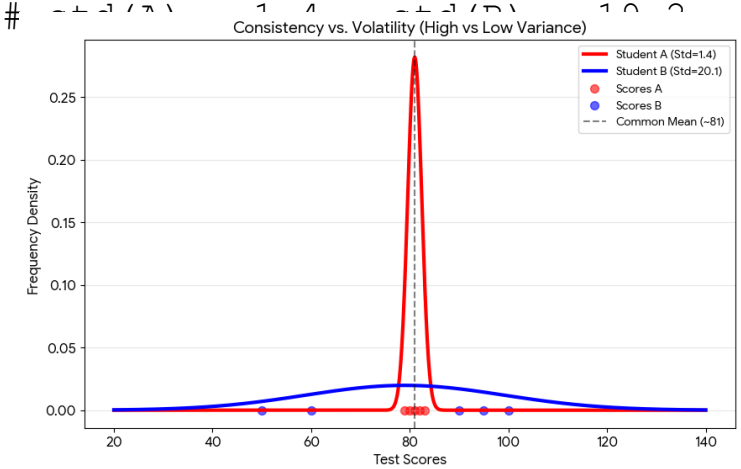
- Phase 2 -- Weeks 2-4: Machine Learning
  - Regression, Classification, Deep Learning & Vision (PyTorch)

- Phase 3 -- Weeks 5-7: AI & LLMs
  - Language Models, RAG (LlamaIndex), AI Agents (smolagents)

- Phase 4 -- Weeks 8-11: Cloud Engineering
  - Azure: VMs, ML Services, and ETL Pipelines

Describing Data: Central Tendency & Spread

```
import numpy as np

# One outlier changes everything
data = [10, 20, 30, 40, 1000]
print('Mean:', np.mean(data)) # 220.0
print('Median:', np.median(data)) #
    30.0
print('Std:', np.std(data)) # 387.3

# Two students, same average
student_A = [80, 82, 81, 79, 83]
student_B = [60, 95, 50, 90, 100]
# std(A) = 1.4   std(B) = 19.2
```

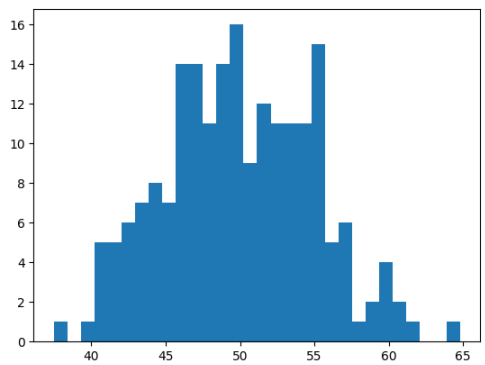


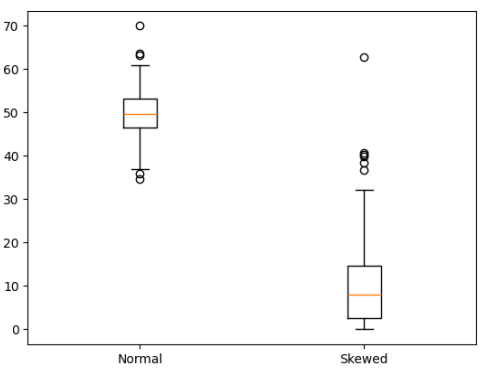
- Mean: pulled by outliers (220 vs 30!)
- Median: stays near center, robust to skew
  - Use median when data is skewed or has outliers
- Std dev: spread in original units
  - Variance: squared units, less interpretable
- Same average, very different story:
  - std(A) = 1.4 (consistent)
  - std(B) = 19.2 (wildly variable)
- groupby().agg(['mean','std']):
  - compute stats per subgroup

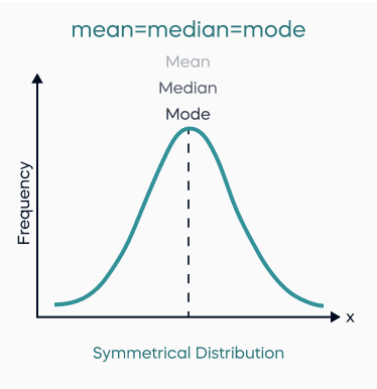
Distributions & Visualization

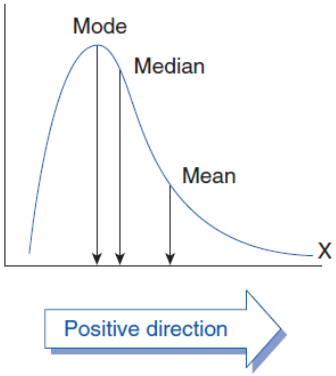
```
import numpy as np
import matplotlib.pyplot as plt

normal = np.random.normal(50, 5, 200)
skewed = np.random.exponential(10, 200)
# Histogram shows shape
plt.hist(normal, bins=30)
# Boxplot compares groups
plt.boxplot([normal, skewed],
            labels=["Normal", "Skewed"])
```

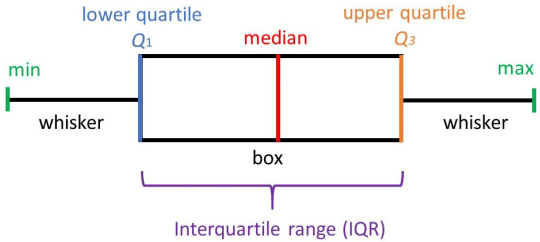
- Normal: symmetric, mean ≈ median ≈ mode
- Right-skewed: long right tail, mean > median
  - Exponential: classic right-skew (wait times)
- Histogram: frequency per bin -- shows shape
- Boxplot anatomy:
  - Box = IQR (Q1 to Q3)
  - Whiskers extend to 1.5 × IQR
  - Dots beyond whiskers = outliers
  - Unequal whiskers → skewness











Hypothesis Testing: The Framework

- H_0 (null): 'nothing is happening' -- our default assumption
- H_1 (alternative): 'something real is going on'

- p-value: probability of this result if H_0 were true
- alpha = 0.05: our threshold for 'surprising enough'

```
p < 0.05  →  reject H0, support H1
p ≥ 0.05  →  not enough evidence to reject H0
```

- Courtroom analogy: innocent (H_0) until strong evidence
  - shows otherwise

- Always write plain-language conclusions,
  - not just 'we reject the null hypothesis'

The t-test in Practice

```
from scipy import stats

class_a = [65, 70, 68, 72, 66, 64, 69,
    71]
class_b = [75, 78, 74, 80, 76, 79, 77,
    81]

t_stat, p_val = stats.ttest_ind(
    class_a, class_b)

if p_val < 0.05:
    print("Significant difference.")
```

- Three versions:
  - Independent samples: two unrelated groups

    ```
    stats.ttest_ind(group_a,
      group_b)
    ```
  - Paired: same subjects, measured twice

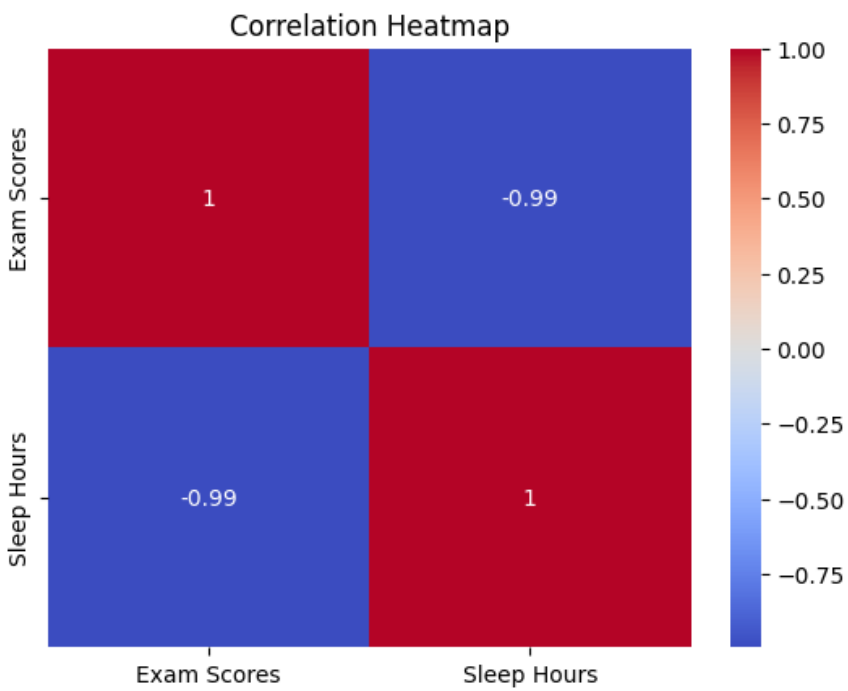
    ```
    stats.ttest_rel(before,
      after)
    ```
  - One-sample: compare to a known value

    ```
    stats.ttest_1samp(scores,
      benchmark)
    ```

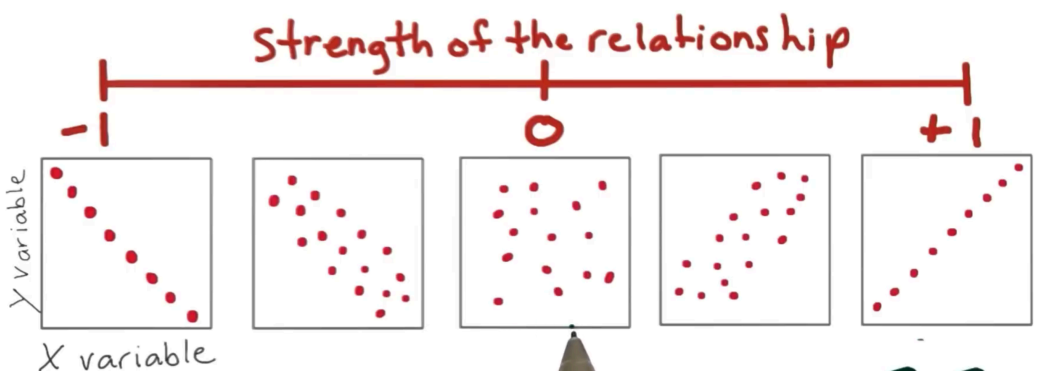
- Two-tailed (default): tests either direction
- t-stat: difference relative to variability

Correlation

```
from scipy.stats import pearsonr
import seaborn as sns

exam_scores = [48, 55, 58, 67, 72, 74, 81, 84]
sleep_hours = [8.2, 7.8,7.3, 6.7, 6.2, 5.8, 5.4,
    4.9]

r, p = pearsonr(exam_scores, sleep_hours)
print('r:', round(r, 2))   # -0.99
print('p:', round(p, 4))   # 0.0

# For a full dataset:
corr = df.corr()
```



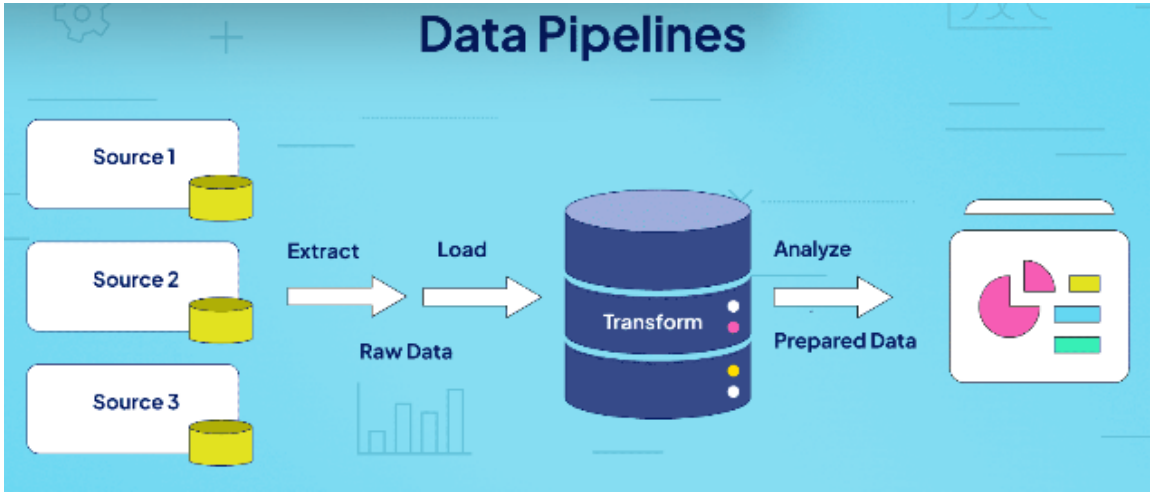
- r ranges from -1 to +1:
  - +1: perfect positive linear relationship
  - -1: perfect negative linear relationship
  - 0: no linear relationship
- Correlation ≠ causation
  - Ice cream & drowning: both peak in summer
- Pearson measures linear relationships only
- Anscombe's Quartet: same r, very different data
  - Always visualize -- never trust r alone



Data Pipelines: The Mindset

```
Raw Data → Load → Clean → Analyze → Visualize → Report
```

- Without pipelines: manual re-runs, brittle scripts, hard to reproduce
- With pipelines: modular steps, one-command runs, scalable to cloud
- ETL: Extract → Transform → Load
  - Pull from source, clean and reshape, store for analysis
- Plain Python functions are a pipeline too
- Add Prefect when you need:
  - retries, structured logging, scheduling, a dashboard



Prefect: Tasks & Flows

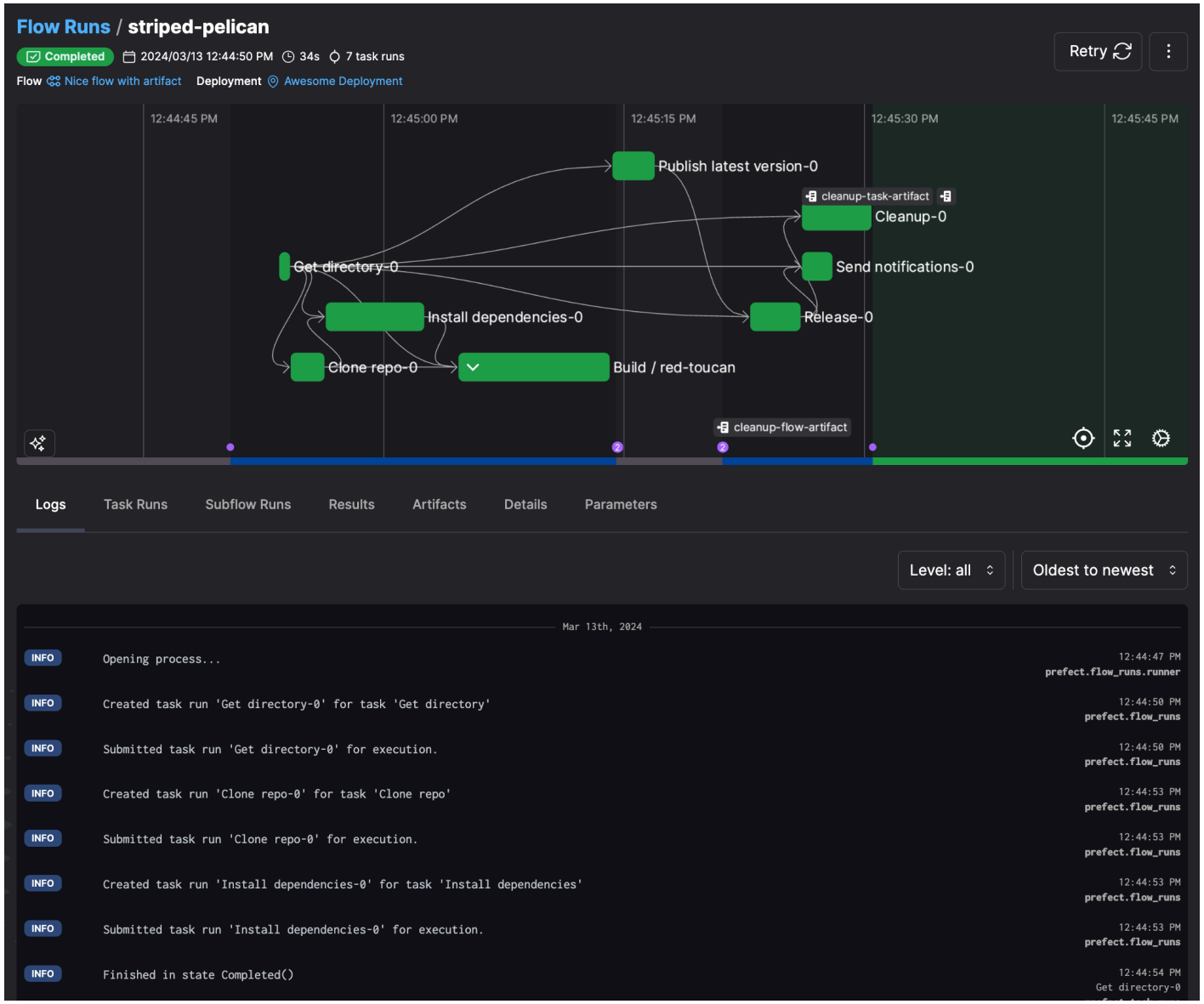
```
from prefect import task, flow

@task
def say_hello(name):
    print(f'Hello, {name}!')

@task
def say_goodbye(name):
    print(f'Goodbye, {name}!')

@flow
def chatty_pipeline(name):
    say_hello(name)
    say_goodbye(name)

if __name__ == '__main__':
    chatty_pipeline('Code the Dream')
```

- @task: atomic unit of work
  - Prefect tracks its state (running / done / failed)
- @flow: orchestrates tasks, single entry point
- Run it like any script: python hello_prefect.py

- Don't panic at the output:
  - Prefect spins up a temporary local server
  - You'll see 'Starting server on 127.0.0.1:...'
  - That's normal -- it's tracking your run

- You get logging, state tracking, run history
  - for free, just by adding two decorators

Prefect: Tasks, Flows & Logs



Adding Real Features

```
@task(retries=3, retry_delay_seconds=2)
def load_data():
    logger = get_run_logger()
    logger.info('Loading data...')
    return pd.read_csv('data.csv')

@task
def clean_data(df):
    logger = get_run_logger()
    df = df.dropna()
    logger.info(f'{len(df)} rows kept')
    return df

# analyze(), report() defined similarly
@flow
def data_pipeline():
    df       = load_data()
    clean_df = clean_data(df)
    results  = analyze(clean_df)
    report(results)
```

- get_run_logger() replaces print():
  - timestamped, stored in UI, filterable
  - INFO / WARNING / ERROR log levels
- retries=3, retry_delay_seconds=2:
  - file I/O fails intermittently in production
  - let Prefect handle it automatically
- Each task has one clear responsibility
- The @flow reads like a recipe

```
prefect server start  ->
    localhost:4200
```

  - watch tasks run live in the dashboard

Week 1 Wrap-up

- What we covered:
  - Describing data: mean, median, std, distributions, histograms, boxplots
  - Hypothesis testing: H0/H1, p-value, t-tests, plain-language conclusions
  - Correlation: Pearson r, heatmaps, correlation ≠ causation
  - Pipelines: ETL mindset, Prefect tasks/flows, logging, retries

- Key ideas to carry forward:
  - Always look at your data before running statistics
  - $p < 0.05$ is a threshold, not a guarantee -- explain results in plain language
  - Correlation reveals relationships; it doesn't explain them
  - Pipelines make analysis reproducible -- one command, every time

- Up next -- Week 2: Machine Learning
  - Regression, scikit-learn, and your first predictive model